

Java. Логування та тестування

Лекція 4

Зміст

- **Логування в Java**
 - Навіщо потрібне логування?
 - Рівні логування
 - Популярні фреймворки (Log4j, SLF4J, Logback)
 - Практичні приклади
- **Юніт-тестування в Java**
 - Що таке юніт-тести?
 - JUnit – основи
 - Assertions та Best Practices
 - Параметризовані тести
 - Mockito для мокування
- **Інтеграція логування та тестування**

Ситуація: Користувач скаржиться, що його платіж не пройшов, але гроші списалися з картки.

Не знаємо, що сталося

Доведеться гадати або відтворювати проблему вручну

Можливо, проблему взагалі не вдасться знайти

З логуванням

```
2025-09-25T14:23:15.412+03:00 INFO [PaymentService] User 12345 initiated payment of 1000 UAH
```

```
2025-09-25T14:23:15.630+03:00 DEBUG [PaymentService] Calling bank API...
```

```
2025-09-25T14:23:17.104+03:00 INFO [BankAPI] Payment successful, transaction ID: TXN789
```

```
2025-09-25T14:23:22.118+03:00 ERROR [PaymentService] Failed to update order status: DatabaseTimeout
```

```
2025-09-25T14:23:22.121+03:00 WARN [PaymentService] Payment completed but order not updated for  
user 12345
```

Знаходимо проблему – гроші списалися, але через `timeout` бази даних замовлення не оновилося. Швидко виправляємо статус вручну і фіксуємо баг.

2025-09-25T14:23:15.412+03:00 INFO [http-nio-8080-exec-7] c.e.p.web.PaymentController trace=4f2a1c8e9b3d5a70 - Received payment request orderId=ORD-88211 userId=12345 amount=1000.00 currency=UAH idempotencyKey=8f1c...a29

2025-09-25T14:23:15.418+03:00 DEBUG [http-nio-8080-exec-7] c.e.p.service.PaymentService trace=4f2a1c8e9b3d5a70 - Acquired payment lock orderId=ORD-88211 in 4ms

2025-09-25T14:23:15.421+03:00 INFO [http-nio-8080-exec-7] c.e.p.service.PaymentService trace=4f2a1c8e9b3d5a70 - Payment initiated paymentId=PAY-4471 orderId=ORD-88211 provider=MONO

2025-09-25T14:23:15.630+03:00 DEBUG [http-nio-8080-exec-7] c.e.p.client.BankApiClient trace=4f2a1c8e9b3d5a70 - POST https://api.bank.example/v2/payments paymentId=PAY-4471 timeout=10000ms attempt=1/3

2025-09-25T14:23:15.633+03:00 DEBUG [http-nio-8080-exec-3] c.e.p.service.PaymentService trace=b71e0d4a2c6f8913 - Payment initiated paymentId=PAY-4472 orderId=ORD-88212 provider=MONO

2025-09-25T14:23:16.902+03:00 DEBUG [http-nio-8080-exec-7] o.a.h.impl.conn.PoolingHttpClient trace=4f2a1c8e9b3d5a70 - Connection released, pool stats: leased=3 available=17 max=20

2025-09-25T14:23:17.104+03:00 INFO [http-nio-8080-exec-7] c.e.p.client.BankApiClient trace=4f2a1c8e9b3d5a70 - Bank response status=200 result=SUCCESS transactionId=TXN789 paymentId=PAY-4471 duration=1474ms

2025-09-25T14:23:17.108+03:00 INFO [http-nio-8080-exec-7] c.e.p.service.PaymentService trace=4f2a1c8e9b3d5a70 - Payment authorized paymentId=PAY-4471 transactionId=TXN789 state=PAID

2025-09-25T14:23:22.118+03:00 ERROR [http-nio-8080-exec-7] c.e.p.service.OrderStatusUpdater trace=4f2a1c8e9b3d5a70 - Failed to update order status orderId=ORD-88211 paymentId=PAY-4471 targetState=PAID attempt=1/3 elapsed=5010ms

org.springframework.dao.QueryTimeoutException: JDBC query timed out after 5000ms; SQL [UPDATE orders SET status = ? WHERE id = ?]

at o.s.jdbc.support.SQLTimeoutExceptionTranslator.translate(SQLTimeoutExceptionTranslator.java:64)

at o.s.jdbc.core.JdbcTemplate.update(JdbcTemplate.java:1032)

at c.e.p.repository.OrderRepository.updateStatus(OrderRepository.java:87)

at c.e.p.service.OrderStatusUpdater.markAsPaid(OrderStatusUpdater.java:41)

... 47 common frames omitted

Caused by: java.sql.SQLException: Statement cancelled due to timeout or client request

at com.mysql.cj.jdbc.StatementImpl.executeUpdate(StatementImpl.java:1348)

... 51 common frames omitted

2025-09-25T14:23:22.121+03:00 WARN [http-nio-8080-exec-7] c.e.p.service.PaymentService trace=4f2a1c8e9b3d5a70 - Inconsistent state: payment captured but order not updated orderId=ORD-88211 paymentId=PAY-4471 transactionId=TXN789 userId=12345 action=enqueued_for_reconciliation

2025-09-25T14:23:22.124+03:00 INFO [http-nio-8080-exec-7] c.e.p.recon.ReconciliationQueue trace=4f2a1c8e9b3d5a70 - Task enqueued taskId=REC-10042 type=ORDER_STATUS_SYNC orderId=ORD-88211 retryAfter=60s

2025-09-25T14:23:22.131+03:00 INFO [http-nio-8080-exec-7] c.e.p.web.PaymentController trace=4f2a1c8e9b3d5a70 - Request completed status=202 orderId=ORD-88211

Вступ

Логування – це процес запису інформації про роботу програми під час її виконання.

Це структурований спосіб фіксувати події, помилки, попередження та іншу важливу інформацію, яка допомагає розробникам розуміти, що відбувається в додатку.

На відміну від `System.out.println()`, професійне логування дає можливість:

- Контролювати рівень деталізації логів
- Записувати інформацію в різні місця (файли, бази даних, консоль)

- Фільтрувати логи за рівнями важливості
- Форматувати вивід для зручного читання та аналізу

`System.out.println()` vs логування

- `println` – йде лише в консоль, зникає після перезапуску
- Логування – зберігається, структуроване, має рівні важливості, можна налаштовувати

Основні цілі

Troubleshooting – знаходження помилок у production

- Коли користувач повідомляє про помилку, логи допомагають відтворити ситуацію
- Можна побачити послідовність викликів та стан даних

Моніторинг – відстеження роботи додатку в реальному часі

- Скільки запитів обробляється
- Чи є проблеми з підключенням до БД
- Error rate – скільки відсотків запитів падає

Аудит – запис важливих подій (хто, що, коли зробив)

- Фінансові операції
- Зміни в налаштуваннях системи
- Дії адміністраторів

Аналіз продуктивності – виявлення bottleneck'ів

- Які операції виконуються довго
- Де витрачається найбільше ресурсів

Рівні логування

Стандартні рівні логування (від найменш до найбільш критичного):

TRACE(600) – найдетальніша інформація, кожен крок виконання

DEBUG(500) – діагностична інформація для розробників

INFO(400) – важливі події в нормальній роботі програми

WARN(300) – попередження, щось працює не оптимально, але не критично

ERROR(200) – помилки, які потребують уваги

FATAL(100) – критичні помилки, програма не може продовжувати роботу

Числові значення визначають пріоритет рівня. Коли встановлено рівень логування (наприклад, **INFO**), система показує всі повідомлення з цим рівнем і вище (**INFO**, **WARN**, **ERROR**, **FATAL**). Це дозволяє фільтрувати логи за важливістю.

TRACE Entering method `calculateDiscount()` with params:
`price=100, discount=10%`

DEBUG Database query executed in 45ms: `SELECT * FROM
users WHERE id=123`

INFO User '`john@example.com`' successfully logged in

WARN Database connection pool is 80% full, consider
scaling

ERROR Failed to send email to `user@example.com`:
`SMTPException`

FATAL Cannot connect to database, application shutting
down

Спрощений приклад: без дат, потоків і traceId



Controller Layer:

```
INFO [UserController] POST /api/users/12345/profile - Request received  
DEBUG [UserController] Request fields: [email, phone]
```

Service Layer:

```
INFO [UserService] Updating profile for user ID: 12345  
DEBUG [UserService] Validating user data: email=n***@example.com,  
phone=+38050***4567
```

Repository/DAO Layer:

```
DEBUG [UserRepository] Executing query: UPDATE users SET email=?, phone=? WHERE  
id=?  
TRACE [UserRepository] Query parameters: [n***@example.com, +38050***4567, 12345]  
DEBUG [UserRepository] Query executed successfully in 23ms  
INFO [UserRepository] User 12345 updated successfully
```

Service Layer:

```
INFO [UserService] Profile update completed for user 12345 fields=[email, phone]
```

Controller Layer (Response):

```
INFO [UserController] POST /api/users/12345/profile - Success (200 OK) -  
Response time: 87ms
```



Controller Layer:

INFO [UserController] POST /api/users/12345/profile - Request received

DEBUG [UserController] Request fields: [email, phone]

Service Layer:

INFO [UserService] Updating profile for user ID: 12345

DEBUG [UserService] Validating user data: email=n***@example.com,
phone=+38050***4567

Repository/DAO Layer:

DEBUG [UserRepository] Executing query: UPDATE users SET email=?, phone=? WHERE id=?

ERROR [UserRepository] DataIntegrityViolationException: constraint=uk_users_email
field=email userId=12345

org.springframework.dao.DataIntegrityViolationException: ...

... 47 more

Service Layer:

WARN [UserService] Profile update failed userId=12345 reason=EMAIL_ALREADY_EXISTS

Controller Layer (Response):

WARN [UserController] POST /api/users/12345/profile - Client error (409
Conflict) - 45ms

Популярні фреймворки

1. `java.util.logging (JUL)`

- Вбудований в Java з версії 1.4, не потребує додаткових залежностей
- Простий у використанні для базових потреб
- Обмежені можливості конфігурації
- Рідко використовується в *enterprise*-проектах через слабку функціональність
- Конфігурується через файл `logging.properties`

2. `Log4j 2`

- Повна переписана версія `Log4j 1.x`
- Асинхронне логування для високої продуктивності
- Гнучка конфігурація через `XML`, `JSON`, `YAML` або програмно
- Підтримка різних *appender*'ів (консоль, файли, бази даних, *Kafka*, тощо)
- Автоматичне перезавантаження конфігурації без перезапуску додатку

- Проблеми з безпекою в старих версіях (`Log4Shell vulnerability`)

3. `Logback`

- Нативна підтримка `SLF4J`
- Автоматичне стиснення та архівування старих логів
- Умовна обробка в конфігурації
- Конфігурація через `XML` файл (`logback.xml`)

4. `SLF4J (Simple Logging Facade for Java)`

- Це абстракція/фасад, а не реальна реалізація логування
- Дозволяє писати код незалежно від конкретного фреймворку
- Можна легко змінити `Log4j` на `Logback` без зміни коду
- Підтримує параметризовані повідомлення для кращої продуктивності
- Де-факто стандарт в Java-спільноті

- Типова зв'язка: `SLF4J (API) + Logback/Log4j2 (Implementation)`

```
import java.util.logging.Level;
import java.util.logging.Logger;
```

nataliia

```
public class GroupFileParser {
    private static final Logger logger = Logger.getLogger(GroupFileParser.class.getName());

    /** Parses a single CSV line into a Group object. ...*/

    public static Group parseGroupFromLine(String line) throws InvalidDataException {...}

    /** Reads groups from a CSV file. ...*/

    public static List<Group> parseFromCSV(String filePath) throws IOException, InvalidDataException {
        List<Group> groups = new ArrayList<>();
        Path path = Path.of(filePath);

        if (!Files.exists(path)) {
            throw new IOException("File not found: " + filePath);
        }

        logger.log(Level.INFO, msg: "Starting to parse groups from file: {0}", filePath);
```

```
import org.slf4j.LoggerFactory;
import ua.university.exception.InvalidDataException;
import ua.university.util.SubjectUtils;
```

nataliia *

```
public record Subject(String name, int credits) implements Comparable<Subject> {
```

```
    private static final Logger logger = LoggerFactory.getLogger(Subject.class);
```

nataliia *

```
public Subject {
    String trimmedName = name != null ? name.trim() : null;

    if (!SubjectUtils.isValidName(trimmedName)) {
        String errorMsg = "Invalid subject name: " + name + "";
        logger.error(errorMsg);
        throw new InvalidDataException(errorMsg);
    }

    if (!SubjectUtils.isValidCredit(credits)) {
        String errorMsg = "Invalid credit amount: " + credits + " (must be 1-5)";
        logger.error(errorMsg);
        throw new InvalidDataException(errorMsg);
    }

    name = trimmedName;
    logger.info("Subject created successfully: {} with {} credits", name, credits);
}
```

Конфігурація

```
<configuration>
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
    </encoder>
  </appender>

  <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
    <file>app.log</file>
    <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
      <fileNamePattern>app.%d{yyyy-MM-dd}.log</fileNamePattern>
      <maxHistory>30</maxHistory>
      <totalSizeCap>3GB</totalSizeCap>
    </rollingPolicy>
    <encoder>
      <pattern>%d{yyyy-MM-dd HH:mm:ss} %-5level %logger - %msg%n</pattern>
    </encoder>
  </appender>

  <logger name="com.myapp" level="DEBUG"/>

  <root level="INFO">
    <appender-ref ref="CONSOLE" />
    <appender-ref ref="FILE" />
  </root>
```

Best practices

- Що логувати:

- ☒ Важливі бізнес-події (створення замовлення, оплата)
- ☒ Помилки та виключення з контекстом
- ☒ Початок/завершення критичних операцій
- ☒ Конфіденційні дані (паролі, токени, номери карток)
- ☒ Надто багато `DEBUG` в `production`

- Як логувати:

- Не дублюйте логи – якщо `Exception` логується в `Repository`, не треба логувати його знову в `Service`
- Логуйте на правильному рівні (не все `ERROR`, не все `INFO`)

- Додавайте контекст: не просто `"Error"`, а `"Failed to save user 12345 to database"`

- Продуктивність:

- Для ресурсномістких операцій додавайте перевірку рівня: `if (logger.isDebugEnabled()) { logger.debug(...); }`
- Уникайте `String concatenation` в параметрах логування. Завжди використовуйте параметризацію `logger.info("User {}", username)` замість `logger.info("User " + username)`

- Структура:

- Включайте `timestamp`, `thread`, `logger name`, рівень
- Використовуйте змістовні назви логерів (зазвичай ім'я класу)

Unit тестування в Java

Що таке юніт-тести?

- Юніт-тест — це автоматичний тест, який перевіряє роботу невеликої одиниці коду (зазвичай одного методу або класу) ізольовано від інших частин системи.
- Основні характеристики:
 - Тестує одну конкретну функціональність
 - Швидко виконується (мілісекунди)
 - Не залежить від зовнішніх систем (БД, API, файли)
 - Повторюваний — завжди однаковий результат
- Навіщо потрібні:
 - Впевненість, що код працює правильно
 - Легше знаходити баги
 - Безпечний рефакторинг
 - Документація коду (тести показують, як користуватися класом)

JUnit - ОСНОВИ

Що таке JUnit?

- Найпопулярніший фреймворк для юніт-тестування в Java
- Поточна версія - JUnit 6 (Jupiter)
- Надає анотації, `assertions` та інструменти для запуску тестів

Основні анотації:

- `@Test` - позначає метод як тестовий
- `@BeforeEach` - виконується перед кожним тестом
- `@AfterEach` - виконується після кожного тесту
- `@BeforeAll` - виконується один раз перед всіма тестами (статичний метод, якщо не вказано `@TestInstance(PER_CLASS)`)
- `@AfterAll` - виконується один раз після всіх тестів (статичний метод)

- `@DisplayName` - задає зрозумілу назву тесту
- `@Disabled` - вимикає тест

```
class CalculatorTest {  
  
    @Test  
    void shouldAddTwoNumbers() {  
        // Arrange (підготовка)  
        Calculator calc = new Calculator();  
  
        // Act (дія)  
        int result = calc.add(2, 3);  
  
        // Assert (перевірка)  
        assertEquals(5, result);  
    }  
}
```

Assertions та Best Practices

Основні Assertions з JUnit 5:

- `assertEquals(expected, actual)` – перевірка рівності
- `assertTrue(condition)` / `assertFalse(condition)` – перевірка `boolean`
- `assertNull(object)` / `assertNotNull(object)` – перевірка на `null`
- `assertThrows(Exception.class, () -> {...})` – перевірка виключень
- `assertAll()` – групування assertions

Best Practices:

- Один тест – один концепт (одна причина впасти)
- Зрозумілі назви тестів (що тестується і очікуваний результат)
- Arrange-Act-Assert патерн
- Тести мають бути незалежними один від одного
- Не використовуйте логіку в тестах (`if`, `loops`)
- Додавайте повідомлення до `assertion` там, де з виводу неясно, що саме зламалося – особливо для `assertTrue/assertFalse`

assertAll vs множинні assert

Проблема з множинними assert:

```
@Test
void testUserValidation() {
    User user = new User("John", "john@example.com", 25);

    assertEquals("John", user.getName());    // Якщо
fail – тест зупиняється тут
    assertEquals("john@example.com", user.getEmail()); //
Це не виконається
    assertEquals(25, user.getAge());          // І це теж
}
```

Якщо перша перевірка провалиться – ми не побачимо результати інших перевірок.

Рішення – assertAll():

```
@Test
void testUserValidation() {
    User user = new User("John", "john@example.com", 25);

    assertAll("User validation",
        () -> assertEquals("John", user.getName()),
        () -> assertEquals("john@example.com",
user.getEmail()),
        () -> assertEquals(25, user.getAge())
    );
}
```

Переваги assertAll:

Виконує ВСІ перевірки, навіть якщо якась провалилася

Показує всі помилки одразу

Параметризовані тести

Дозволяє запустити один тест з різними наборами даних, замість дублювання коду.

Основні джерела даних:

@ValueSource – прості значення одного типу:

```
@ParameterizedTest
@ValueSource(ints = {1, 2, 3, 5, 8})
void shouldBePositive(int number) {
    assertTrue(number > 0);
}
```

@CsvSource – кілька параметрів через кому:

```
@ParameterizedTest
@CsvSource({
    "2, 3, 5",
    "10, 5, 15",
    "-2, 2, 0"
})
void shouldAddNumbers(int a, int b, int expected) {
```

```
    assertEquals(expected, calculator.add(a, b));
}
```

@MethodSource – складні об'єкти через метод:

```
@ParameterizedTest
@MethodSource("provideUserData")
void shouldValidateUser(String name, String email,
    boolean isValid) {
    assertEquals(isValid, validator.isValid(name,
    email));
}
```

```
static Stream<Arguments> provideUserData() {
    return Stream.of(
        Arguments.of("John", "john@example.com",
    true),
        Arguments.of("", "invalid", false)
    );
}
```

Заглушки

Що таке **Mock**?

- Імітація (заглушка) реального об'єкта для тестування
- Дозволяє ізолювати тестований клас від залежностей
- Можна контролювати поведінку та перевіряти виклики методів

Навіщо потрібно:

- Тестувати клас без реальної БД, API, файлової системи

- Симулювати різні сценарії (помилки, тайм-аути)
- Швидкі тести без зовнішніх залежностей

Основні можливості **Mockito**:

- Створення **mock**-об'єктів
- Налаштування поведінки (**stubbing**)
- Перевірка викликів (**verification**)

Приклад

```
class UserServiceTest {  
  
    @Mock  
    private Logger logger;  
  
    @Mock  
    private UserRepository userRepository;  
  
    @InjectMocks  
    private UserService userService;  
  
    @BeforeEach  
    void setUp() {  
        MockitoAnnotations.openMocks(this);  
    }  
  
    @Test  
    void shouldLogErrorWhenUserNotFound() {  
        // Arrange
```

```
        when(userRepository.findById(123)).thenReturn(null);  
  
        // Act  
        userService.getUser(123);  
  
        // Assert – перевіряємо що ERROR лог був  
        викликаний  
        verify(logger).error("User not found: {}",  
            123);  
    }  
}
```

@Mock – створює mock об'єкт

@InjectMocks – створює тестований об'єкт і
впроваджує в нього mock'и

verify() – перевіряє, що метод був викликаний з
певними параметрами

Інтеграція логування та тестування

При тестуванні:

- Використовуйте окремий конфіг логування для тестів (`test/resources/logback-test.xml`)
- Встановлюйте рівень `ERROR` або `OFF` для тестів, щоб не засмічувати консоль
- Не перевіряйте логування в кожному тесті – це не бізнес-логіка

При розробці:

- Додавайте логування на етапі написання коду, а не після
- `DEBUG` логи допомагають при написанні тестів
- Якщо тест падає – додайте логування для швидшої діагностики

Загальні поради:

- Логування та тести доповнюють один одного
- Логи допомагають знайти проблему в *production*
- Тести гарантують, що код працює правильно